

CONTENTS

Jerarquía de Memoria y Caché	
Guía detallada desde cero — con los detalles que salvan el parcial	
0. Por qué existe este problema	
1. Principio de localidad	
2. Terminología y tiempo promedio de acceso (AMAT)	
2.1 Extensión a múltiples niveles (L1 + L2 + memoria)	
3. Descomposición de una dirección de memoria: TAG / LÍNEA (o CONJUNTO) / BYTE	
3.1 Correspondencia directa	
3.2 Correspondencia totalmente asociativa	
3.3 Correspondencia asociativa por conjuntos (N vías)	
3.4 Tabla resumen — campos según tipo de correspondencia	
4. Algoritmos de reemplazo	
5. Políticas de escritura: Write Through vs. Write Back	
6. Coherencia de caché en sistemas multiprocesador	
7. Ejemplo completo: hit rate según orden de recorrido de una matriz	
8. Ejercicios de repaso rápido (con respuesta)	
9. Resumen ultra-rápido (2 minutos)	

Jerarquía de Memoria y Caché

GUÍA DETALLADA DESDE CERO — CON LOS DETALLES QUE SALVAN EL PARCIAL

Arquitectura de Computadoras — FING UdelaR

Localidad temporal/espacial · Hit/Miss/AMAT · Correspondencia directa/asociativa/por conjuntos · LRU/FIFO/LFU/Random · Write Through/Write Back · Coherencia de caché · Recorrido de matrices

Cómo usar esta guía: este es el tema teórico corto más recurrente en los exámenes analizados (9 de 13). Cada sección tiene definición corta, procedimiento paso a paso, al menos dos ejemplos numéricos resueltos y verificados, tabla resumen donde aplica, y un recuadro rojo con el error que más se paga en el parcial. Todos los cálculos de bits fueron verificados sumando TAG+LÍNEA/CONJUNTO+BYTE y confirmando que dan el total de bits de dirección del enunciado; todos los cálculos de tiempo/hit rate fueron verificados numéricamente.

0. Por qué existe este problema

Un procesador es rapidísimo: puede ejecutar una instrucción por ciclo (o más). La memoria principal (RAM) es muchísimo más lenta que el procesador, pero es grande y barata. Una memoria tan rápida como el procesador (SRAM) existe, pero es carísima y solo se puede fabricar en tamaños pequeños.

Esto plantea un dilema económico, no solo técnico: **no podés tener simultáneamente memoria grande, barata y rápida**. Tenés que elegir dos de las tres.

La solución de la industria es no elegir: se arma una **jerarquía** de varios niveles de memoria, cada uno con un compromiso distinto:

- **Registros** — brutalmente rápidos, carísimos por bit, muy pocos (decenas).
- **Caché (L1, L2, L3)** — rápida, cara, chica (KB a pocos MB).
- **Memoria principal (RAM)** — más lenta, más barata, más grande (GB).
- **Disco/almacenamiento secundario** — lentísimo, baratísimo, enorme (TB).

La caché es el “colchón” entre el procesador (rapidísimo) y la RAM (lenta): guarda una copia de los datos usados recientemente para que la mayoría de los accesos se resuelvan a velocidad de caché en vez de tener que esperar a la RAM. Que esto funcione **no es magia ni azar**: funciona porque los programas reales no acceden a la memoria de forma aleatoria, sino que exhiben **principio de localidad** (Sección 1). Sin localidad, la caché sería inútil.

Idea que hay que tener siempre presente: la caché no es “una copia completa” de la RAM — es mucho más chica. Toda la teoría de este módulo (correspondencia, reemplazo, escritura, coherencia) existe para responder una sola pregunta operativa: **dado un bloque de memoria, ¿en qué línea de la caché puede estar, y cómo lo sé?**

1. Principio de localidad

Es la propiedad empírica (observada en programas reales) que hace que la caché funcione. Tiene dos formas:

Localidad temporal: si un dato fue accedido, es probable que se vuelva a acceder **pronto** (de nuevo, en poco tiempo). Ejemplo típico: una variable contador o un acumulador que se lee y escribe en cada vuelta de un bucle.

Localidad espacial: si se accedió a una dirección de memoria, es probable que **pronto** se acceda a direcciones **cercanas** a esa. Ejemplo típico: recorrer un arreglo de forma secuencial — acceder a `a[i]` hace probable que enseguida se acceda a `a[i+1]`.

Ejemplo verificado (Práctico 11, Ejercicio 1) — código:

```
for (i = 0; i < 20; i++)
  for (j = 0; j < 10; j++)
    a[i] = a[i] * a[j]
```

- **Localidad temporal:** para un `i` fijo, la celda `a[i]` se lee y escribe en cada una de las 10 vueltas del bucle interno (`j`) — se vuelve a referenciar 10 veces seguidas en poco tiempo.
- **Localidad espacial:** a medida que `j` avanza (0,1,2,...,9), se accede a `a[j]`, `a[j+1]`, `a[j+2]` ... es decir, a posiciones de memoria consecutivas — clásico recorrido secuencial de arreglo.

Ejemplo 2 — búsqueda binaria (poca localidad temporal, buena localidad espacial):

en la búsqueda binaria sobre un arreglo ordenado, el espacio de búsqueda se reduce a la mitad en cada iteración, por lo que casi nunca se vuelve a acceder a una posición ya visitada poco tiempo después (**escasa localidad temporal**). Sin embargo, en cada iteración se accede a `a[medio]` y luego se ajustan los límites `izq`/`der`, que quedan cerca de posiciones ya visitadas — hay **localidad espacial moderada/alta** porque el algoritmo sigue trabajando dentro de una región del arreglo que se va acotando.

Error frecuente: confundir “localidad espacial” con “acceso secuencial estricto”. No hace falta que el acceso sea `i, i+1, i+2...` exactamente — alcanza con que las direcciones accedidas estén **cerca** unas de otras en un intervalo corto de tiempo. La búsqueda binaria de arriba tiene localidad espacial sin ser secuencial.

2. Terminología y tiempo promedio de acceso (AMAT)

Hit (acierto): el dato buscado **está** en la caché. **Miss (fallo):** el dato buscado **no está** en la caché, hay que ir a buscarlo a un nivel más lento (memoria principal, u otro nivel de caché). **Hit rate:** proporción de accesos que son hit. $\text{hit_rate} = \text{hits} / \text{accesos_totales}$. **Miss rate:** proporción de accesos que son miss. $\text{miss_rate} = 1 - \text{hit_rate}$. **Hit time:** tiempo (en ciclos) que tarda la caché en responder cuando hay hit. **Miss penalty:** tiempo (en ciclos) **adicional** que hay que esperar cuando hay miss, para traer el bloque del nivel siguiente (más lento) y recién ahí completar el acceso.

Fórmula del tiempo promedio de acceso a memoria (AMAT — Average Memory Access Time):

$$\text{AMAT} = \text{hit_time} + \text{miss_rate} \times \text{miss_penalty}$$

La lógica: siempre pagás el hit_time (todo acceso empieza consultando la caché). Además, con probabilidad miss_rate , pagás un costo extra de miss_penalty para resolver el fallo.

Ejemplo 1 (verificado): $\text{hit_time} = 1$ ciclo, $\text{miss_rate} = 5\% = 0.05$, $\text{miss_penalty} = 100$ ciclos.

$$\text{AMAT} = 1 + 0.05 \times 100 = 1 + 5 = \mathbf{6 \text{ ciclos.}}$$

Verificación con Python: $1 + 0.05 * 100 = 6.0$ ✓

Ejemplo 2 (verificado): hit_time = 2 ciclos, miss_rate = 2% = 0.02, miss_penalty = 150 ciclos.

AMAT = $2 + 0.02 \times 150 = 2 + 3 = 5$ ciclos.

Verificación con Python: `2 + 0.02*150 = 5.0` ✓

2.1 EXTENSIÓN A MÚLTIPLES NIVELES (L1 + L2 + MEMORIA)

Con dos niveles de caché, el “miss penalty” de L1 es en realidad “ir a buscar a L2”, que a su vez puede tener hit o miss:

$$AMAT = HT_{L1} + MR_{L1} \times (HT_{L2} + MR_{L2} \times T_{\text{memoria}})$$

donde MR_{L2} es la tasa de fallo **local** de L2 (de los accesos que efectivamente le llegan a L2, es decir, los que ya fallaron en L1).

Ejemplo verificado: $HT_{L1}=1$ ciclo, $MR_{L1}=5\%$, $HT_{L2}=10$ ciclos, $MR_{L2}=20\%$, $T_{\text{memoria}}=100$ ciclos.

AMAT = $1 + 0.05 \times (10 + 0.20 \times 100) = 1 + 0.05 \times (10+20) = 1 + 0.05 \times 30 = 1 + 1.5 = 2.5$ ciclos.

Verificación con Python: `1 + 0.05*(10 + 0.20*100) = 2.5` ✓

Error frecuente de examen: usar la tasa de fallo **global** de L2 (fallos de L2 respecto al total de accesos de la CPU) en vez de la **local** (fallos de L2 respecto a los accesos que le llegaron, es decir, los que ya fallaron en L1). Si el enunciado da la tasa global, hay que dividirla por MR_{L1} antes de usarla en esta fórmula, o usar directamente la fórmula con tasas globales: $AMAT = HT_{L1} + MR_{L1} \times HT_{L2} + MR_{\text{global},L2} \times T_{\text{memoria}}$. Leé siempre con cuidado qué tasa te están dando.

3. Descomposición de una dirección de memoria: TAG / LÍNEA (o CONJUNTO) / BYTE

Toda dirección de memoria de n bits se interpreta, para efectos de la caché, dividida en campos. El procedimiento general (para cualquier tipo de correspondencia) es:

1. Calcular el **tamaño de línea** en bytes → determina cuántos bits necesita el campo **BYTE** (desplazamiento dentro de la línea): `byte_bits = log2(tamaño_línea)`.

- Según el tipo de correspondencia, calcular cuántos bits necesita el campo de **LÍNEA** o **CONJUNTO** (Secciones 3.1 a 3.3).
- Los bits que **sobran** de los n bits totales van al **TAG**.
- Verificar siempre:** TAG_bits + LÍNEA_bits (o CONJUNTO_bits) + BYTE_bits = n (bits totales de dirección).

El TAG es lo que se guarda junto a cada línea de caché para saber “a qué bloque de memoria principal corresponde esta línea que tengo guardada acá”. Al buscar una dirección, se usa el campo LÍNEA/CONJUNTO para saber **dónde mirar**, y ahí se compara el TAG almacenado contra el TAG de la dirección buscada: si coinciden, hit; si no, miss.

3.1 CORRESPONDENCIA DIRECTA

Definición: cada bloque de memoria principal tiene **una única línea posible** de la caché donde puede alojarse (relación 1 a 1... en realidad muchos bloques de memoria compiten por la misma línea, pero cada bloque solo tiene **una** línea candidata). El campo LÍNEA de la dirección **es** el índice de esa línea: $\text{índice_línea} = (\text{dirección} / \text{tamaño_línea}) \bmod \text{cantidad_de_líneas}$.

Como cada bloque tiene un único destino posible, **no hace falta ninguna política de reemplazo**: si la línea candidata está ocupada por otro bloque, simplemente se lo desaloja y se pone el nuevo (no hay “elección” que hacer).

Ejemplo 1 (verificado, Práctico 11 Ej.3a): caché de 4 KB, 256 líneas, direcciones de 20 bits.

Tamaño de línea = 4096 bytes / 256 líneas = 16 bytes/línea = $2^4 \rightarrow$ **BYTE = 4 bits**.

256 líneas = $2^8 \rightarrow$ **LÍNEA = 8 bits**.

TAG = 20 - 4 - 8 = 8 bits.

Verificación: $8 + 8 + 4 = 20$ bits ✓ (coincide con los 20 bits de dirección del enunciado).

Esquema: `TAG[19..12] | LÍNEA[11..4] | BYTE[3..0]`

Ejemplo 2 (verificado, Práctico 11 Ej.5a): caché de 128 KB, líneas de 32 bytes, direcciones de 32 bits, correspondencia directa.

128 KB = 2^{17} bytes. Línea = 32 bytes = $2^5 \rightarrow$ **BYTE = 5 bits**.

Cantidad de líneas = $2^{17} / 2^5 = 2^{12} = 4096$ líneas $\rightarrow$ **LÍNEA = 12 bits**.

TAG = 32 - 5 - 12 = 15 bits.

Verificación: $15 + 12 + 5 = 32$ bits ✓.

Esquema: `TAG[31..17] | LÍNEA[16..5] | BYTE[4..0]`

Error frecuente: calcular la cantidad de líneas dividiendo el tamaño de caché por el tamaño de línea, pero después olvidarse de convertir esa cantidad a **bits** (aplicando $\log_2$) antes de restarla del total. El campo **LÍNEA** no “vale” la cantidad de líneas, vale la cantidad de **bits** necesarios para numerarlas.

3.2 CORRESPONDENCIA TOTALMENTE ASOCIATIVA

Definición: cualquier bloque de memoria puede ir a **cualquier línea** de la caché — no hay una línea fija candidata. Por eso, **no existe campo LÍNEA** en la dirección: solo hay **TAG y BYTE**. Al buscar un dato, hay que comparar el TAG buscado contra el TAG almacenado en **todas** las líneas de la caché (en paralelo, con comparadores de hardware, o sería demasiado lento) — de ahí el nombre “totalmente asociativa”.

Como cualquier bloque puede ir a cualquier línea, cuando la caché está llena **sí hace falta una política de reemplazo** para decidir qué línea desalojar (Sección 4).

Ejemplo 1 (verificado, Práctico 11 Ej.3b): misma caché que el ejemplo anterior — 4 KB, 256 líneas, direcciones de 20 bits, pero ahora totalmente asociativa.

Línea = 16 bytes = 2^4 → **BYTE = 4 bits** (esto no cambia, depende solo del tamaño de línea).

No hay campo **LÍNEA** → **TAG = 20 - 4 = 16 bits**.

Verificación: **16 + 4 = 20 bits** ✓.

Esquema: `TAG[19..4] | BYTE[3..0]`

Ejemplo 2 (verificado): CPU de direcciones de 24 bits, caché L1 totalmente asociativa de 1 KB con 16 líneas.

Tamaño de línea = 1024 bytes / 16 líneas = 64 bytes = 2^6 → **BYTE = 6 bits**.

No hay campo **LÍNEA** → **TAG = 24 - 6 = 18 bits**.

Verificación: **18 + 6 = 24 bits** ✓.

Esquema: `TAG[23..6] | BYTE[5..0]`

Error frecuente: escribir un campo “**LÍNEA**” en el esquema de una caché totalmente asociativa. No existe — es justamente la característica que la distingue de la directa. Si tu esquema tiene 3 campos en una totalmente asociativa, está mal.

3.3 CORRESPONDENCIA ASOCIATIVA POR CONJUNTOS (N VÍAS)

Definición: solución intermedia entre las dos anteriores. Las líneas se agrupan en **conjuntos** de N líneas cada uno (N = cantidad de “vías”). Un bloque de memoria tiene un único **conjunto** candidato (como en la directa), pero **dentro** de ese conjunto puede ir a **cualquiera** de las N líneas (como en la totalmente asociativa, pero acotado a esas N).

$\text{cantidad_de_conjuntos} = \text{cantidad_de_líneas} / N$. El campo de la dirección se llama **CONJUNTO** (no LÍNEA): $\text{conjunto_bits} = \log_2(\text{cantidad_de_conjuntos})$.

Al buscar, se usa CONJUNTO para saber en qué conjunto mirar, y ahí se comparan los N TAGs almacenados contra el TAG buscado. Si hay miss y el conjunto está lleno, hace falta política de reemplazo, pero solo hay que elegir entre las N líneas de ese conjunto (mucho más barato en hardware que comparar contra **todas** las líneas de la caché).

Ejemplo 1 (verificado, Práctico 11 Ej.3c): misma caché — 4 KB, 256 líneas, direcciones de 20 bits, asociativa por conjuntos de **2 vías**.

Línea = 16 bytes → **BYTE = 4 bits**.

Conjuntos = 256 líneas / 2 vías = 128 conjuntos = 2^7 → **CONJUNTO = 7 bits**.

TAG = 20 - 4 - 7 = 9 bits.

Verificación: 9 + 7 + 4 = 20 bits ✓.

Esquema: `TAG[19..11] | CONJUNTO[10..4] | BYTE[3..0]`

Ejemplo 2 (verificado, Práctico 11 Ej.4): microprocesador de direcciones de 32 bits, caché on-chip de 32 KB, asociativa por conjuntos de **4 vías**, línea de 4 palabras de 32 bits (=16 bytes).

32 KB = 2^{15} bytes. Línea = 16 bytes = 2^4 → **BYTE = 4 bits**.

Cantidad de líneas = $2^{15} / 2^4 = 2^{11} = 2048$ líneas.

Conjuntos = 2048 líneas / 4 vías = 512 conjuntos = 2^9 → **CONJUNTO = 9 bits**.

TAG = 32 - 4 - 9 = 19 bits.

Verificación: 19 + 9 + 4 = 32 bits ✓.

Esquema: `TAG[31..13] | CONJUNTO[12..4] | BYTE[3..0]`

Aplicación completa del Ejemplo 2 — decodificar la dirección 0xABCDE8F8:

En binario (32 bits): 1010 1011 1100 1101 1110 1000 1111 1000

Separando según el esquema TAG[31..13] | CONJUNTO[12..4] | BYTE[3..0]:

TAG = 1010101111001101111 (19 bits)

CONJUNTO = 010001111 (9 bits) = $128+8+4+2+1 = 143$

BYTE = 1000 (4 bits) = 8

Verificado con Python: `CONJUNTO=143, BYTE=8` ✓ (coincide con el ejercicio resuelto de práctico).

Conclusión: la palabra de esa dirección se busca en el **conjunto 143**, comparando el TAG contra las 4 líneas (vías) de ese conjunto, en el **byte 8** de la línea que dé match.

Error frecuente: confundir “vías” con “conjuntos”. Si el enunciado dice “4 vías” y “2048 líneas”, el número de conjuntos es $2048/4=512$, **no** 2048. El campo de la dirección siempre se calcula sobre la cantidad de **conjuntos**, nunca sobre la cantidad total de líneas cuando hay más de 1 vía.

3.4 TABLA RESUMEN — CAMPOS SEGÚN TIPO DE CORRESPONDENCIA

Correspondencia	Campos de la dirección	¿Campo LÍNEA/CONJUNTO?	Búsqueda del TAG	¿Necesita política de reemplazo?
Directa	TAG, LÍNEA, BYTE	Sí (índice único)	Se compara 1 solo TAG (el de esa línea)	No
Totalmente asociativa	TAG, BYTE	No existe	Se comparan todos los TAG de la caché	Sí
Asociativa por N vías	TAG, CONJUNTO, BYTE	Sí (índice de conjunto)	Se comparan los N TAG del conjunto	Sí

Truco para no perderse: cuantas más “vías” tiene una asociativa por conjuntos, más se parece a una totalmente asociativa (más comparaciones en paralelo, menos bits de CONJUNTO, más bits de TAG). El caso extremo $N=1$ vía es exactamente la correspondencia directa; el caso extremo “ N = cantidad total de líneas” (1 solo conjunto) es exactamente la totalmente asociativa.

4. Algoritmos de reemplazo

Se necesitan solo cuando un bloque tiene **más de una** línea candidata (asociativa total o por conjuntos). Cuando hay que desalojar una línea para hacer lugar a un bloque nuevo, hay que elegir cuál:

- **LRU (Least Recently Used — menos usado recientemente)**: desaloja la línea que hace más tiempo que no se accede. Es la que mejor aprovecha la localidad temporal (si algo no se usó hace rato, es razonable asumir que no se va a necesitar pronto), por lo que suele dar el mejor hit rate en la práctica. **Contra**: hay que llevar la cuenta exacta de “cuándo fue el último acceso” de cada línea del conjunto, lo cual es costoso de implementar en hardware real (crece con el número de vías) — por eso en conjuntos grandes se usan aproximaciones a LRU en vez de LRU exacto.
- **FIFO (First In, First Out)**: desaloja la línea que **entró primero** al conjunto, sin importar si se usó después o no. Mucho más simple de implementar (un simple puntero circular), pero puede desalojar una línea que se sigue usando activamente.
- **LFU (Least Frequently Used — menos frecuentemente usado)**: desaloja la línea con **menor cantidad de accesos totales** (un contador de frecuencia, no de recencia). Distinto de LRU: una línea puede haberse usado hace mucho pero muchas veces (LFU la conserva), o haberse usado hace poquito pero solo una vez (LRU la conserva, LFU no necesariamente).
- **Random**: se elige una línea al azar del conjunto para desalojar. Sorprendentemente competitivo en la práctica (evita patrones patológicos) y trivial de implementar en hardware, pero no aprovecha ninguna información sobre el comportamiento real del programa.

Por qué LRU “suele ganar” pero es caro: como aprovecha directamente el principio de localidad temporal (Sección 1), en la mayoría de los programas reales predice mejor qué línea no hace falta que las otras tres. El costo en hardware es la razón por la que procesadores reales con muchas vías por conjunto suelen implementar variantes aproximadas (pseudo-LRU) en vez de LRU exacto.

Error frecuente: decir que la correspondencia **directa** “usa FIFO” o “usa LRU”. No usa ninguna política de reemplazo — no hay elección posible, cada bloque tiene una única línea destino (Sección 3.1).

5. Políticas de escritura: Write Through vs. Write Back

Definen qué pasa cuando la CPU **escribe** un dato que está en caché (hit de escritura).

Write Through (“a través”): la escritura se hace **simultáneamente** en la caché **y** en la memoria principal, siempre.

- Ventaja: la memoria principal siempre está actualizada y consistente con la caché — más simple de razonar, y trivial para coherencia multiprocesador.
- Desventaja: cada escritura paga el costo de acceso a memoria principal (lenta), aunque haya sido hit en caché — más lento.

Write Back (“de vuelta” / diferida): la escritura se hace **solo en la caché**. Se marca la línea con un bit adicional llamado **dirty** (o “modificado”). La memoria principal se actualiza recién **cuando esa línea se desaloja** de la caché (y solo si el bit dirty está en 1 — si nunca se modificó, no hace falta escribirla de vuelta).

- Ventaja: mucho más rápido, porque las escrituras repetidas a la misma línea no generan tráfico a memoria principal cada vez.
- Desventaja: necesita el bit dirty extra, y la memoria principal puede quedar “desactualizada” respecto a la caché durante un tiempo — hay que tener más cuidado en sistemas con múltiples procesadores o con DMA.

	Write Through	Write Back
¿Cuándo escribe a memoria principal?	En cada hit de escritura	Solo al desalojar una línea modificada
¿Necesita bit dirty?	No	Sí
Velocidad de escritura	Más lenta (memoria en el camino crítico)	Más rápida
Consistencia con memoria principal	Siempre consistente	Puede estar desactualizada temporalmente
Tráfico hacia memoria principal	Mayor (una escritura por cada hit)	Menor (una escritura por línea desalojada, agrupando varias modificaciones)

Error frecuente: decir que Write Back “nunca escribe a memoria principal”. Sí escribe — pero de forma diferida, solo al desalojar una línea que tiene el bit dirty en 1. Si la línea nunca se modificó (dirty=0), al desalojarla no hace falta escribirla porque memoria principal ya tiene ese mismo valor.

6. Coherencia de caché en sistemas multiprocesador

El problema: en un sistema con varios procesadores (o núcleos), cada uno con su propia caché, puede pasar que **dos cachés distintas tengan copias del mismo bloque de memoria**. Si un procesador modifica su copia (sobre todo con Write Back, donde la modificación puede quedar solo en su caché por un buen rato), las otras cachés quedan con una copia **desactualizada** de ese mismo dato sin saberlo — esto es el **”problema de coherencia de caché”**.

Solución típica: Bus Snooping (“espionaje de bus”). Cada caché tiene lógica adicional que **escucha permanentemente** el bus de memoria compartido, observando todas las transacciones de escritura que hacen las demás cachés/procesadores. Si una caché detecta (por el bus) que otro procesador escribió un bloque que ella también tiene cacheado, **invalida su propia copia** de ese bloque (la marca como inválida), forzando que la próxima vez que se necesite ese dato haya un miss y se traiga la versión actualizada.

Idea intuitiva: es como si cada caché tuviera un “oído” pegado al bus común, y cada vez que escucha “alguien escribió el bloque X”, revisa si tiene una copia de X — si la tiene, la tira (la invalida) para no arriesgarse a usar un dato viejo.

Error frecuente: pensar que Bus Snooping actualiza automáticamente la copia invalidada con el nuevo valor. No la actualiza — la **invalida**. La actualización ocurre recién cuando ese procesador vuelva a necesitar el dato y genere un nuevo miss (o, en protocolos más sofisticados tipo MESI, mediante mecanismos de transferencia caché-a-caché, que exceden el alcance de esta introducción).

7. Ejemplo completo: hit rate según orden de recorrido de una matriz

Este es un ejercicio clásico de examen: comparar el hit rate de recorrer una matriz **por filas** (orden en que está almacenada en memoria — *row-major*) contra recorrerla **por columnas**.

Montaje del ejemplo (verificado por simulación en Python):

- Matriz de enteros de 4×4 ($N=4$), cada entero de 4 bytes, almacenada por filas (*row-major*): la fila 0 completa ocupa direcciones consecutivas, luego la fila 1, etc.
- Línea de caché = 16 bytes = exactamente 4 enteros = **exactamente una fila completa entra en una línea**.

- Caché de correspondencia directa con **2 líneas** (para poder mostrar el efecto de conflicto): fila i mapea a la línea $i \bmod 2$.

Recorrido por filas (for i: for j: acceder $a[i][j]$): para cada fila, el primer acceso ($j=0$) es miss (trae la línea completa a caché) y los 3 accesos siguientes ($j=1,2,3$) son hit porque ya están en la misma línea recién cargada. Como nunca se vuelve a una fila después de terminarla, no hay conflictos.

- Total de accesos: 16. Misses: 4 (uno por fila). Hits: 12.
- **Hit rate = $12/16 = 75\%$**

Recorrido por columnas (for j: for i: acceder $a[i][j]$): para $j=0$ se accede a $a[0][0]$, $a[1][0]$, $a[2][0]$, $a[3][0]$ — es decir, filas 0,1,2,3 en secuencia, cada una en una línea distinta. Como solo hay 2 líneas físicas y las filas 0 y 2 comparten la misma línea (línea 0), y las filas 1 y 3 comparten la otra (línea 1), acceder a la fila 2 **desaloja** a la fila 0 (que recién se había cargado), y acceder a la fila 3 desaloja a la fila 1. Al pasar a $j=1$ hay que volver a empezar por la fila 0, que ya fue desalojada → miss de nuevo. Esto se repite en cada columna.

- Total de accesos: 16. Misses: **16** (todos). Hits: **0**.
- **Hit rate = $0/16 = 0\%$**

Verificación por simulación (Python, cache directa de 2 líneas simulada accediendo en cada orden): recorrido por filas → hits=12, misses=4, hit rate=75.00%. Recorrido por columnas → hits=0, misses=16, hit rate=0.00%. Diferencia total: 75 puntos porcentuales, con la misma matriz y la misma caché — la única variable es el **orden de acceso**.

Por qué pasa esto: el recorrido por filas coincide con el orden real de almacenamiento en memoria (row-major), así que aprovecha al máximo la localidad espacial: cada línea traída se consume completa antes de pasar a la siguiente. El recorrido por columnas salta entre filas lejanas en cada paso, sin aprovechar nada de lo que ya se cargó en la línea — de hecho, en este ejemplo llega a desperdiciar cada línea cargada (activa un miss, usa solo 1 de sus 4 datos, y la línea se descarta antes de reusarla).

Con una matriz mucho más grande que la caché (el caso realista de un examen, y del Práctico 11 Ejercicio 5, con matrices de 512×512), el fenómeno es el mismo mecanismo pero más extremo: recorrer por filas da hit rate alto (limitado solo por cuántos elementos entran en cada línea), mientras que recorrer por columnas puede degenerar en **miss en cada acceso** si la caché no alcanza a retener una fila completa hasta la próxima vez que se necesita.

Error frecuente: decir “si guardo la matriz por columnas en vez de por filas, el recorrido por columnas mejora”. Es cierto, pero solo porque estás **invirtiendo el layout de memoria para que vuelva a coincidir con el orden de acceso** — el principio de fondo no cambia: lo que importa siempre es que el orden de acceso coincida con el orden de almacenamiento contiguo en memoria, sea cual sea ese orden.

8. Ejercicios de repaso rápido (con respuesta)

Pregunta	Respuesta
Caché de 8 KB, líneas de 32 bytes, correspondencia directa, direcciones de 24 bits. ¿Bits de BYTE, LÍNEA y TAG?	BYTE=5 ($32=2^5$), líneas= $8192/32=256=2^8 \rightarrow$ LÍNEA=8, TAG=24-5-8=11. Verificación: $11+8+5=24 \checkmark$
Misma caché pero totalmente asociativa. ¿Bits de TAG?	No hay campo LÍNEA $\rightarrow$ TAG=24-5=19. Verificación: $19+5=24 \checkmark$
Misma caché, asociativa por conjuntos de 4 vías. ¿Bits de CONJUNTO y TAG?	conjuntos= $256/4=64=2^6 \rightarrow$ CONJUNTO=6, TAG=24-5-6=13. Verificación: $13+6+5=24 \checkmark$
hit_time=1 ciclo, miss_rate=10%, miss_penalty=50 ciclos. ¿AMAT?	AMAT= $1+0.10 \times 50=1+5=6$ ciclos
¿Qué política de reemplazo no aplica nunca en correspondencia directa?	Ninguna aplica — no hay elección posible, cada bloque tiene una única línea destino
¿Cuál escritura necesita bit “dirty”?	Write Back
¿Cuál escritura es más lenta pero mantiene memoria principal siempre actualizada?	Write Through
¿Qué hace una caché al detectar por Bus Snooping que otro procesador escribió un bloque que también tiene cacheado?	Invalida su propia copia de ese bloque
Recorrer una matriz almacenada por filas, ¿conviene recorrerla por filas o por columnas para maximizar hit rate?	Por filas (coincide con el orden de almacenamiento — mejor localidad espacial)
LRU vs FIFO: ¿cuál tiene en cuenta cuándo fue el último acceso, y cuál solo el orden de llegada?	LRU tiene en cuenta el último acceso (recencia); FIFO solo el orden de llegada, sin importar el uso posterior

9. Resumen ultra-rápido (2 minutos)

- **Por qué existe la caché:** rápido+grande+barato es imposible a la vez $\rightarrow$ jerarquía de niveles, cada uno con su compromiso.

- **Localidad temporal:** se reusa pronto lo mismo. **Localidad espacial:** se usa pronto algo cercano. Sin localidad, la caché no sirve de nada.
- **AMAT = hit_time + miss_rate × miss_penalty.** Con 2 niveles: $AMAT = HT_1 + MR_1 \times (HT_2 + MR_2 \times T_{mem})$, con MR_2 local.
- **Dirección = TAG + (LÍNEA o CONJUNTO) + BYTE.** BYTE depende del tamaño de línea. Directa → tiene LÍNEA (única candidata, sin reemplazo). Totalmente asociativa → sin campo de línea, TAG grande, busca en todas partes, sí necesita reemplazo. Por conjuntos de N vías → CONJUNTO fijo, N líneas candidatas dentro, sí necesita reemplazo. **Siempre verificar que los bits sumen el total de la dirección.**
- **Reemplazo:** LRU (mejor hit rate, más caro), FIFO (simple, ignora uso), LFU (por frecuencia), Random (simple, competitivo). Solo aplica si hay más de una línea candidata.
- **Escritura:** Write Through = escribe siempre en caché y memoria (lento, siempre consistente). Write Back = escribe solo en caché con bit dirty, vuelca a memoria al desalojar (rápido, requiere dirty).
- **Coherencia multiprocesador:** Bus Snooping = cada caché escucha el bus y **invalida** (no actualiza) su copia si detecta que otro procesador escribió ese bloque.
- **Matriz por filas vs columnas:** recorrer en el mismo orden en que está almacenada en memoria (row-major → por filas) maximiza localidad espacial y hit rate; recorrerla “cruzado” puede degenerar en miss casi constante.